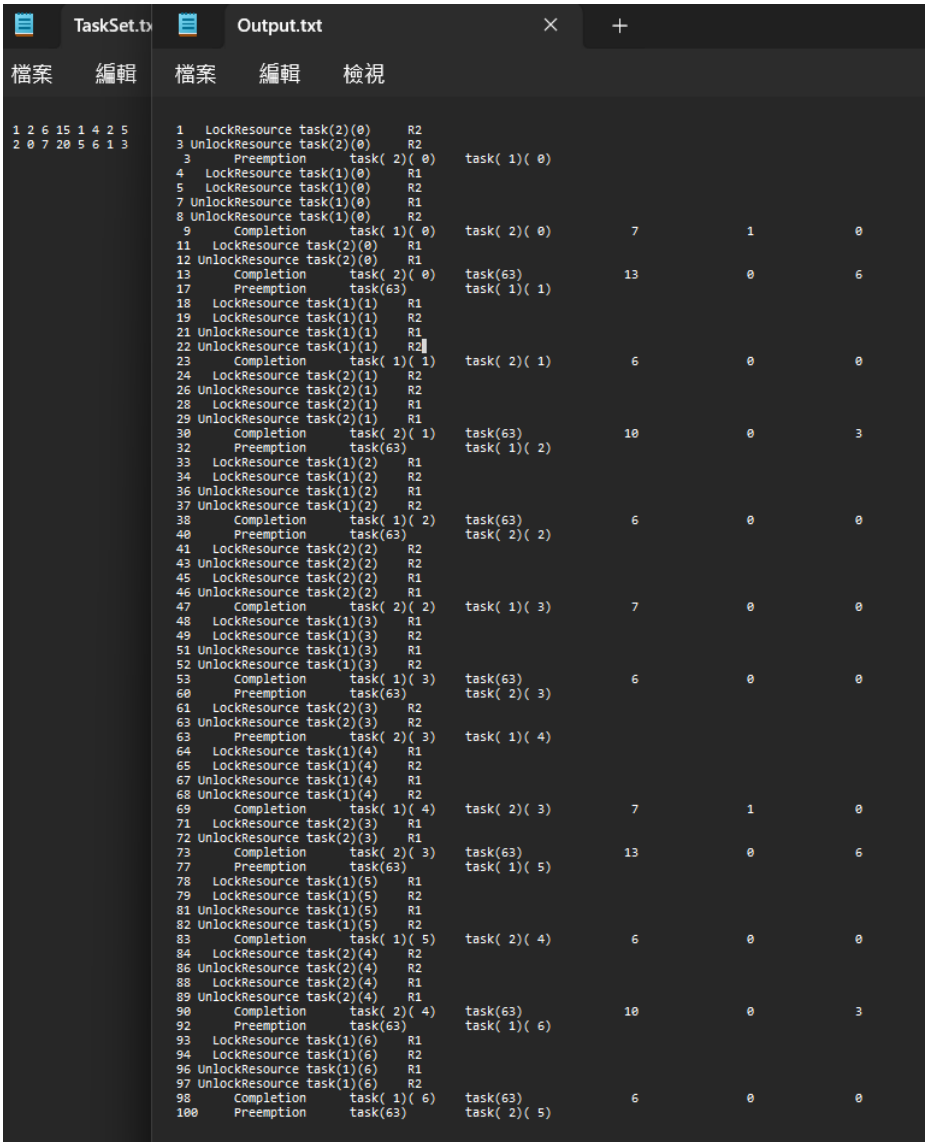
Embedded OS Implementation
Spring 2025 Project #3

M11302149 電子碩一 趙孟哲

[PART I] NPCS Implementation [45%]

A. Results of examples. (35%)

Taskset 1:



TaskSet.tx

```
1 2 6 15 1 4 2 5
2 0 7 20 5 6 1 3
```

Output.txt

檔案 編輯 檢視

```
1   LockResource task(2)(0)     R2
3 UnlockResource task(2)(0)     R2
 3      Preemption      task( 2)( 0)    task( 1)( 0)
4   LockResource task(1)(0)     R1
5   LockResource task(1)(0)     R2
7 UnlockResource task(1)(0)     R1
8 UnlockResource task(1)(0)     R2
 9      Completion      task( 1)( 0)    task( 2)( 0)          7             1             0
11   LockResource task(2)(0)    R1
12 UnlockResource task(2)(0)    R1
13      Completion      task( 2)( 0)    task(63)             13             0             6
17      Preemption      task(63)        task( 1)( 1)
18   LockResource task(1)(1)    R1
19   LockResource task(1)(1)    R2
21 UnlockResource task(1)(1)    R1
22 UnlockResource task(1)(1)    R2
23      Completion      task( 1)( 1)    task( 2)( 1)          6             0             0
24   LockResource task(2)(1)    R2
26 UnlockResource task(2)(1)    R2
28   LockResource task(2)(1)    R1
29 UnlockResource task(2)(1)    R1
30      Completion      task( 2)( 1)    task(63)             10             0             3
32      Preemption      task(63)        task( 1)( 2)
33   LockResource task(1)(2)    R1
34   LockResource task(1)(2)    R2
36 UnlockResource task(1)(2)    R1
37 UnlockResource task(1)(2)    R2
38      Completion      task( 1)( 2)    task(63)              6             0             0
40      Preemption      task(63)        task( 2)( 2)
41   LockResource task(2)(2)    R2
43 UnlockResource task(2)(2)    R2
45   LockResource task(2)(2)    R1
46 UnlockResource task(2)(2)    R1
47      Completion      task( 2)( 2)    task( 1)( 3)          7             0             0
48   LockResource task(1)(3)    R1
49   LockResource task(1)(3)    R2
51 UnlockResource task(1)(3)    R1
52 UnlockResource task(1)(3)    R2
53      Completion      task( 1)( 3)    task(63)              6             0             0
60      Preemption      task(63)        task( 2)( 3)
61   LockResource task(2)(3)    R2
63 UnlockResource task(2)(3)    R2
63      Preemption      task( 2)( 3)    task( 1)( 4)
64   LockResource task(1)(4)    R1
65   LockResource task(1)(4)    R2
67 UnlockResource task(1)(4)    R1
68 UnlockResource task(1)(4)    R2
69      Completion      task( 1)( 4)    task( 2)( 3)          7             1             0
71   LockResource task(2)(3)    R1
72 UnlockResource task(2)(3)    R1
73      Completion      task( 2)( 3)    task(63)             13             0             6
77      Preemption      task(63)        task( 1)( 5)
78   LockResource task(1)(5)    R1
79   LockResource task(1)(5)    R2
81 UnlockResource task(1)(5)    R1
82 UnlockResource task(1)(5)    R2
83      Completion      task( 1)( 5)    task( 2)( 4)          6             0             0
84   LockResource task(2)(4)    R2
86 UnlockResource task(2)(4)    R2
88   LockResource task(2)(4)    R1
89 UnlockResource task(2)(4)    R1
90      Completion      task( 2)( 4)    task(63)             10             0             3
92      Preemption      task(63)        task( 1)( 6)
93   LockResource task(1)(6)    R1
94   LockResource task(1)(6)    R2
96 UnlockResource task(1)(6)    R1
97 UnlockResource task(1)(6)    R2
98      Completion      task( 1)( 6)    task(63)              6             0             0
100     Preemption      task(63)        task( 2)( 5)
```

Taskset 2:

TaskSet.txt

```
檔案  編輯  檢視

1 1 8 32 1 6 0 0
2 8 5 30 0 0 0 0
3 0 4 20 0 0 1 3
```

Output.txt

```
檔案  編輯  檢視

1   LockResource task(3)(0)     R2
3 UnlockResource task(3)(0)     R2
 4      Completion      task( 3)( 0)    task( 1)( 0)          4               0               0
5   LockResource task(1)(0)     R1
10 UnlockResource task(1)(0)    R1
10      Preemption      task( 1)( 0)    task( 2)( 0)
15      Completion      task( 2)( 0)    task( 1)( 0)          7               2               0
17      Completion      task( 1)( 0)    task(63)             16               0               8
20      Preemption      task(63)        task( 3)( 1)
21   LockResource task(3)(1)    R2
23 UnlockResource task(3)(1)    R2
24      Completion      task( 3)( 1)    task(63)              4               0               0
33      Preemption      task(63)        task( 1)( 1)
34   LockResource task(1)(1)    R1
39 UnlockResource task(1)(1)    R1
39      Preemption      task( 1)( 1)    task( 2)( 1)
40      Preemption      task( 2)( 1)    task( 3)( 2)
41   LockResource task(3)(2)    R2
43 UnlockResource task(3)(2)    R2
44      Completion      task( 3)( 2)    task( 2)( 1)          4               0               0
48      Completion      task( 2)( 1)    task( 1)( 1)         10               1               4
50      Completion      task( 1)( 1)    task(63)             17               0               9
60      Preemption      task(63)        task( 3)( 3)
61   LockResource task(3)(3)    R2
63 UnlockResource task(3)(3)    R2
64      Completion      task( 3)( 3)    task(63)              4               0               0
65      Preemption      task(63)        task( 1)( 2)
66   LockResource task(1)(2)    R1
71 UnlockResource task(1)(2)    R1
71      Preemption      task( 1)( 2)    task( 2)( 2)
76      Completion      task( 2)( 2)    task( 1)( 2)          8               3               0
78      Completion      task( 1)( 2)    task(63)             13               0               5
80      Preemption      task(63)        task( 3)( 4)
81   LockResource task(3)(4)    R2
83 UnlockResource task(3)(4)    R2
84      Completion      task( 3)( 4)    task(63)              4               0               0
97      Preemption      task(63)        task( 1)( 3)
98   LockResource task(1)(3)    R1
```

B. Description of implementation (10%)

首先分別在 task_para_set 加上紀錄 R1,R2 的欄位；在 ucosii.h 宣告 global variable R1,R2。

```
/*Task structure*/
typedef struct task_para_set {
    INT16U TaskID;
    INT16U TaskArriveTime;
    INT16U TaskExecutionTime;
    INT16U TaskPeriodic;
    INT16U TaskNumber;
    INT16U TaskPriority;
    INT16U R1_LockTime;
    INT16U R1_UnlockTime;
    INT16U R2_LockTime;
    INT16U R2_UnlockTime;

}task_para_set;
int TASK_NUMBER;
/*Task structure*/
```

```
/*resorce*/
INT8S R1_Owner; // -1 means no owner, n means task n is the owner
INT8S R2_Owner; // -1 means no owner, n means task n is the owner
/*resorce*/
```

在 OSTCB 增加 R1,R2 的欄位和 Blocked Time 的欄位。

```
int holding_r1;
int holding_r2;

INT8U R1_LockTime;
INT8U R1_UnlockTime;
INT8U R2_LockTime;
INT8U R2_UnlockTime;

INT32U BlockingTime;
INT32U BlockingStartTick;
int  IsBlocked; // 1: blocked, 0: not blocked
```

在先前 RM scheduling 的基礎下，在遍歷 OSTCBList 時，呼叫 resource_manager()，該函式為管理 R1,R2，會紀錄 lock、unlock 和 holding 資訊，供後面 OS_SchedNew 判斷是否要切換 task。並且印出訊息。

```
void resource_manager(OS_TCB* ptcb) {
    INT32U now = ptcb->execution_time - ptcb->remaining;
    INT32U now_time = OSTimeGet();

    // Lock R1
    if (now == ptcb->R1_LockTime && !ptcb->holding_r1 && ptcb->R1_LockTime !=0) {
        if (R1_Owner == -1) {
            R1_Owner = ptcb->TaskID;
            ptcb->holding_r1 = 1;
            fprintf(Output_fp, "%d   LockResource task(%d)(%d)\tR1\n", now_time, ptcb->TaskID, ptcb->TaskNumber);
            printf("%d   LockResource task(%d)(%d)\tR1\n", now_time, ptcb->TaskID, ptcb->TaskNumber);
        }
    }

    // Unlock R1
    if (now == ptcb->R1_UnlockTime && ptcb->holding_r1 && ptcb->R1_UnlockTime != 0) {
        R1_Owner = -1;
        ptcb->holding_r1 = 0;
        fprintf(Output_fp, "%d UnlockResource task(%d)(%d)\tR1\n", now_time, ptcb->TaskID, ptcb->TaskNumber);
        printf("%d UnlockResource task(%d)(%d)\tR1\n", now_time, ptcb->TaskID, ptcb->TaskNumber);
    }

    // Lock R2
    if (now == ptcb->R2_LockTime && !ptcb->holding_r2 && ptcb->R2_LockTime != 0) {
        if (R2_Owner == -1) {
            R2_Owner = ptcb->TaskID;
            ptcb->holding_r2 = 1;
            fprintf(Output_fp, "%d   LockResource task(%d)(%d)\tR2\n", now_time, ptcb->TaskID, ptcb->TaskNumber);
            printf("%d   LockResource task(%d)(%d)\tR2\n", now_time, ptcb->TaskID, ptcb->TaskNumber);
        }
    }

    // Unlock R2
    if (now == ptcb->R2_UnlockTime && ptcb->holding_r2 && ptcb->R2_UnlockTime != 0) {
        R2_Owner = -1;
        ptcb->holding_r2 = 0;
        fprintf(Output_fp, "%d UnlockResource task(%d)(%d)\tR2\n", now_time, ptcb->TaskID, ptcb->TaskNumber);
        printf("%d UnlockResource task(%d)(%d)\tR2\n", now_time, ptcb->TaskID, ptcb->TaskNumber);
    }
}
```

```
ptcb = OSTCBList;                                    /* Point at first TCB in TCB list               */
while (ptcb->OSTCBPrio != OS_TASK_IDLE_PRIO) {       /* Go through all TCBs in TCB list              */

    OS_ENTER_CRITICAL();
    //check resorce lock
    resource_manager(ptcb);

    //printf("task: %2d remaining : %2d\n", ptcb->TaskID,ptcb->remaining);
    // Deadline miss
    if (ptcb->state == 1 && ptcb->remaining > 0 && OSTime >= ptcb->deadline) {
        ptcb->state = 4;
    }
    // INT8U kkk = OSTime;
    if (ptcb->OSTCBDly != 0u) {                      /* No, Delayed or waiting for event with TO     */
        if(ptcb->state ==0)
            ptcb->OSTCBDly--;                            /* Decrement nbr of ticks to end of delay       */
        if (ptcb->OSTCBDly == 0u && ptcb->state == 0) {               /* Check for timeout
            ptcb->state = 5;
            if ((ptcb->OSTCBStat & OS_STAT_PEND_ANY) != OS_STAT_RDY) {
                ptcb->OSTCBStat &= (INT8U)~(INT8U)OS_STAT_PEND_ANY;   /* Yes, Clear status flag   */
                ptcb->OSTCBStatPend = OS_STAT_PEND_TO;                /* Indicate PEND timeout    */
            }
            else {
                ptcb->OSTCBStatPend = OS_STAT_PEND_OK;
            }

            if ((ptcb->OSTCBStat & OS_STAT_SUSPEND) == OS_STAT_RDY) {  /* Is task suspended?       */
                OSRdyGrp |= ptcb->OSTCBBitY;             /* No,  Make ready          */
                OSRdyTbl[ptcb->OSTCBY] |= ptcb->OSTCBBitX;
                OS_TRACE_TASK_READY(ptcb);
            }
        }
    }

    ptcb = ptcb->OSTCBNext;                          /* Point at next TCB in TCB list                */
    OS_EXIT_CRITICAL();
}
```

接著在 OS_SchedNew 之前，對於所有建立的 OSTCB，如果 R1,R2 被解鎖了，計算他被鎖住的時間。

```
// check if the task is blocked and preempted
for (int i = 0; i < OS_LOWEST_PRIO; i++) {
    OS_TCB* pt = OSTCBPrioTbl[i];
    if (pt != NULL && pt->IsBlocked == 1) {
        if (R1_Owner == -1 && R2_Owner == -1) {
            pt->IsBlocked = 0;
            pt->BlockingTime += OSTimeGet() - pt->BlockingStartTick;
            //printf("tick: %d   Unblock task(%d)(%d) total: %d\n", OSTimeGet(), pt->TaskID, pt->TaskNumber, pt->BlockingTime);
        }
    }
}


 // 2. Scheduler: decide the next task
OS_SchedNew();
OSTCBHighRdy = OSTCBPrioTbl[OSPrioHighRdy];
```

而在 OS_SchedNew()，加上 NPCS 邏輯，如果現在 OSTCBCur 有鎖定任何一個資源，則不切換，然後對於所有比目前任務高優先權的任務，紀錄此時作為被鎖住(blocked)的時間點。然後更新 OSTCBHighRdy 和 OSPrioHighRdy。

```
static  void  OS_SchedNew (void)
{
#if OS_LOWEST_PRIO <= 63u
    INT8U y;

    y = OSUnMapTbl[OSRdyGrp];
    OSPrioHighRdy = (INT8U)((y << 3u) + OSUnMapTbl[OSRdyTbl[y]]);

    // ---------- NPCS: prevent preemption if current task is in critical section ----------
    if (OSTCBCur != NULL && (OSTCBCur->holding_r1 || OSTCBCur->holding_r2) && OSTime != 0) {
        // keep running current task
        OSPrioHighRdy = OSPrioCur;

        // ---------- BLOCKING detection ----------
        for (INT8U i = 0; i < OSPrioCur; i++) {
            OS_TCB* pt = OSTCBPrioTbl[i];
            if (pt != NULL && pt->state == 1 && pt->IsBlocked == 0) {
                pt->IsBlocked = 1;
                pt->BlockingStartTick = OSTimeGet();
                // printf("tick %u Blocking task %d\n", OSTime,pt->TaskID);
            }
        }
    }
    // ---------- update ----------
    OSTCBHighRdy = OSTCBPrioTbl[OSPrioHighRdy];
```

輸出完成訊息時，Preempted Time 用 Response Time – Blocking Time – Execution Time ，即為所求。

```
if (strcmp(type, "Completion")==0)
{
    printf("%2u\tCompletion\t%-12s\t%-12s\t%8u\t%8u\t%7u\n",
        OSTime,
        curr,
        next,
        OSTime - OSTCBCur->ArriveTime,
        OSTCBCur->BlockingTime,
        OSTime - OSTCBCur->ArriveTime - OSTCBCur->BlockingTime - OSTCBCur->execution_time
    );
    fprintf(Output_fp,"%2u\tCompletion\t%-12s\t%-12s\t%8u\t%8u\t%7u\n",
        OSTime,
        curr,
        next,
        OSTime - OSTCBCur->ArriveTime,
        OSTCBCur->BlockingTime,
        OSTime - OSTCBCur->ArriveTime - OSTCBCur->BlockingTime - OSTCBCur->execution_time
    );

}
```

在 app_hook.c 修改 InputFille()能夠讀取 R1,R2 參數欄位。

```c
void InputFile() {
    errno_t err;
    if ((err = fopen_s(&fp, INPUT_FILE_NAME, "r")) == 0) {
        printf("The file 'TaskSet.txt' was opened\n");
    }
    else {
        printf("The file 'TaskSet.txt' was not opened\n");
        return;
    }

    char str[MAX];
    char* ptr;
    char* pTmp = NULL;
    int TaskInfo[INFO];  // Expecting INFO >= 8
    int i, j = 0;
    TASK_NUMBER = 0;

    while (fgets(str, sizeof(str), fp)) {
        if (str[0] == '\n' || str[0] == '\0') continue; // Skip empty lines
        i = 0;
        ptr = strtok_s(str, " \t\r\n", &pTmp);
        while (ptr != NULL && i < INFO) {
            TaskInfo[i++] = atoi(ptr);
            ptr = strtok_s(NULL, " \t\r\n", &pTmp);
        }

        if (i < 8) continue;  // Skip invalid line

        TaskParameter[j].TaskID = TaskInfo[0];
        TaskParameter[j].TaskArriveTime = TaskInfo[1];
        TaskParameter[j].TaskExecutionTime = TaskInfo[2];
        TaskParameter[j].TaskPeriodic = TaskInfo[3];
        TaskParameter[j].TaskPriority = TaskInfo[3];  // RM: smaller period ¡÷ higher priority

        // store priority ªº task ID
        p2id[TaskParameter[j].TaskPriority] = TaskParameter[j].TaskID;

        TaskParameter[j].R1_LockTime = TaskInfo[4];
        TaskParameter[j].R1_UnlockTime = TaskInfo[5];
        TaskParameter[j].R2_LockTime = TaskInfo[6];
        TaskParameter[j].R2_UnlockTime = TaskInfo[7];

        TASK_NUMBER++;
        j++;
    }

    fclose(fp);
}
```

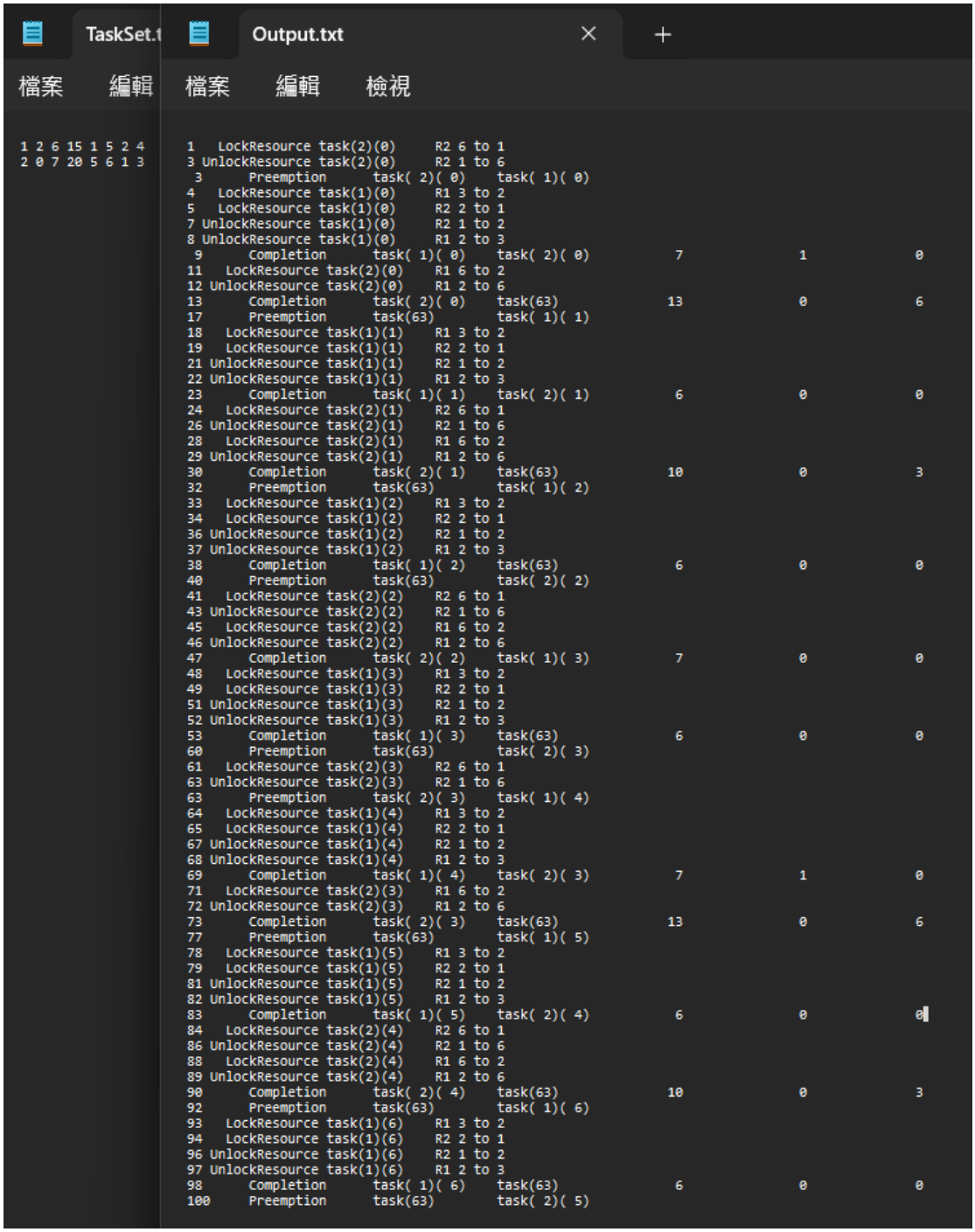
印出完成訊息時，也要重新初始化 blocking time。

```c
if (ptcb->TaskID > 0) {
    switch (ptcb->state) {
    case 2:
        PrintTask("Completion",NULL);
        ptcb->TaskNumber++;
        ptcb->state = 0;

        ptcb->BlockingStartTick = 0;
        ptcb->BlockingTime = 0;
        ptcb->IsBlocked = 0;
        break;
    case 3:
        //printf("%d Arrive task(%d)(job %d) t3\n", OSTime, ptcb->TaskID, ptcb->TaskNumber + 1);
        PrintTask("Completion",NULL);
        ptcb->ArriveTime = OSTime;
        ptcb->remaining = ptcb->execution_time;
        ptcb->TaskNumber++;
        ptcb->deadline = OSTime + ptcb->period;
        ptcb->state = 1;

        ptcb->BlockingStartTick = 0;
        ptcb->BlockingTime = 0;
        ptcb->IsBlocked = 0;

        break;
    case 4:
        //printf("%d MissDeadline task(%d)(job %d)\n", OSTime, ptcb->TaskID, ptcb->TaskNumber);
        Miss_ptcb = ptcb;
        break;
    case 5:
        //printf("%d Arrive task(%d)(job %d)t5\n", OSTime, ptcb->TaskID, ptcb->TaskNumber);
        ptcb->state = 1;
        ptcb->ArriveTime = OSTime;
        ptcb->deadline = OSTime + ptcb->period;
        ptcb->remaining = ptcb->execution_time;

        ptcb->BlockingStartTick = 0;
        ptcb->BlockingTime = 0;
        ptcb->IsBlocked = 0;
        break;
    default:
        break;
    }
}
```

[PART II] CPP Scheduler Implementation [45%]

A. Results of examples. (35%)

Taskset 1:



TaskSet.t

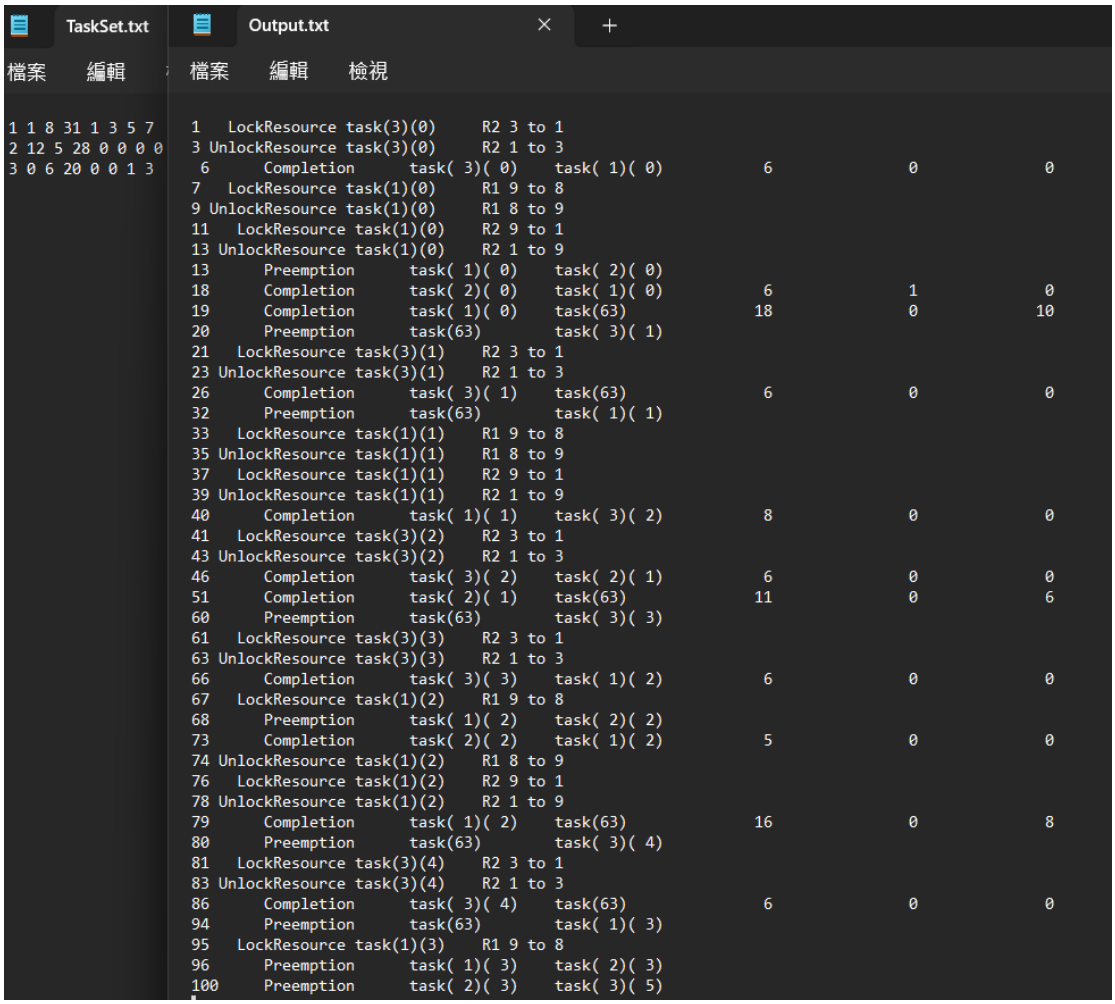
```
1 2 6 15 1 5 2 4
2 0 7 20 5 6 1 3
```

Output.txt

檔案 編輯 檢視

```
1   LockResource task(2)(0)     R2 6 to 1
3 UnlockResource task(2)(0)     R2 1 to 6
 3      Preemption      task( 2)( 0)    task( 1)( 0)
4   LockResource task(1)(0)     R1 3 to 2
5   LockResource task(1)(0)     R2 2 to 1
7 UnlockResource task(1)(0)     R2 1 to 2
8 UnlockResource task(1)(0)     R1 2 to 3
 9      Completion      task( 1)( 0)    task( 2)( 0)          7             1             0
11   LockResource task(2)(0)    R1 6 to 2
12 UnlockResource task(2)(0)    R1 2 to 6
13      Completion      task( 2)( 0)    task(63)             13             0             6
17      Preemption      task(63)        task( 1)( 1)
18   LockResource task(1)(1)    R1 3 to 2
19   LockResource task(1)(1)    R2 2 to 1
21 UnlockResource task(1)(1)    R2 1 to 2
22 UnlockResource task(1)(1)    R1 2 to 3
23      Completion      task( 1)( 1)    task( 2)( 1)          6             0             0
24   LockResource task(2)(1)    R2 6 to 1
26 UnlockResource task(2)(1)    R2 1 to 6
28   LockResource task(2)(1)    R1 6 to 2
29 UnlockResource task(2)(1)    R1 2 to 6
30      Completion      task( 2)( 1)    task(63)             10             0             3
32      Preemption      task(63)        task( 1)( 2)
33   LockResource task(1)(2)    R1 3 to 2
34   LockResource task(1)(2)    R2 2 to 1
36 UnlockResource task(1)(2)    R2 1 to 2
37 UnlockResource task(1)(2)    R1 2 to 3
38      Completion      task( 1)( 2)    task(63)              6             0             0
40      Preemption      task(63)        task( 2)( 2)
41   LockResource task(2)(2)    R2 6 to 1
43 UnlockResource task(2)(2)    R2 1 to 6
45   LockResource task(2)(2)    R1 6 to 2
46 UnlockResource task(2)(2)    R1 2 to 6
47      Completion      task( 2)( 2)    task( 1)( 3)          7             0             0
48   LockResource task(1)(3)    R1 3 to 2
49   LockResource task(1)(3)    R2 2 to 1
51 UnlockResource task(1)(3)    R2 1 to 2
52 UnlockResource task(1)(3)    R1 2 to 3
53      Completion      task( 1)( 3)    task(63)              6             0             0
60      Preemption      task(63)        task( 2)( 3)
61   LockResource task(2)(3)    R2 6 to 1
63 UnlockResource task(2)(3)    R2 1 to 6
63      Preemption      task( 2)( 3)    task( 1)( 4)
64   LockResource task(1)(4)    R1 3 to 2
65   LockResource task(1)(4)    R2 2 to 1
67 UnlockResource task(1)(4)    R2 1 to 2
68 UnlockResource task(1)(4)    R1 2 to 3
69      Completion      task( 1)( 4)    task( 2)( 3)          7             1             0
71   LockResource task(2)(3)    R1 6 to 2
72 UnlockResource task(2)(3)    R1 2 to 6
73      Completion      task( 2)( 3)    task(63)             13             0             6
77      Preemption      task(63)        task( 1)( 5)
78   LockResource task(1)(5)    R1 3 to 2
79   LockResource task(1)(5)    R2 2 to 1
81 UnlockResource task(1)(5)    R2 1 to 2
82 UnlockResource task(1)(5)    R1 2 to 3
83      Completion      task( 1)( 5)    task( 2)( 4)          6             0             0
84   LockResource task(2)(4)    R2 6 to 1
86 UnlockResource task(2)(4)    R2 1 to 6
88   LockResource task(2)(4)    R1 6 to 2
89 UnlockResource task(2)(4)    R1 2 to 6
90      Completion      task( 2)( 4)    task(63)             10             0             3
92      Preemption      task(63)        task( 1)( 6)
93   LockResource task(1)(6)    R1 3 to 2
94   LockResource task(1)(6)    R2 2 to 1
96 UnlockResource task(1)(6)    R2 1 to 2
97 UnlockResource task(1)(6)    R1 2 to 3
98      Completion      task( 1)( 6)    task(63)              6             0             0
100     Preemption      task(63)        task( 2)( 5)
```

Taskset 2:



TaskSet.txt

```
1 1 8 31 1 3 5 7
2 12 5 28 0 0 0 0
3 0 6 20 0 0 1 3
```

Output.txt

```
1   LockResource task(3)(0)     R2 3 to 1
3 UnlockResource task(3)(0)     R2 1 to 3
 6      Completion      task( 3)( 0)    task( 1)( 0)          6               0               0
7   LockResource task(1)(0)     R1 9 to 8
9 UnlockResource task(1)(0)     R1 8 to 9
11   LockResource task(1)(0)    R2 9 to 1
13 UnlockResource task(1)(0)    R2 1 to 9
13      Preemption      task( 1)( 0)    task( 2)( 0)
18      Completion      task( 2)( 0)    task( 1)( 0)          6               1               0
19      Completion      task( 1)( 0)    task(63)             18               0              10
20      Preemption      task(63)        task( 3)( 1)
21   LockResource task(3)(1)    R2 3 to 1
23 UnlockResource task(3)(1)    R2 1 to 3
26      Completion      task( 3)( 1)    task(63)              6               0               0
32      Preemption      task(63)        task( 1)( 1)
33   LockResource task(1)(1)    R1 9 to 8
35 UnlockResource task(1)(1)    R1 8 to 9
37   LockResource task(1)(1)    R2 9 to 1
39 UnlockResource task(1)(1)    R2 1 to 9
40      Completion      task( 1)( 1)    task( 3)( 2)          8               0               0
41   LockResource task(3)(2)    R2 3 to 1
43 UnlockResource task(3)(2)    R2 1 to 3
46      Completion      task( 3)( 2)    task( 2)( 1)          6               0               0
51      Completion      task( 2)( 1)    task(63)             11               0               6
60      Preemption      task(63)        task( 3)( 3)
61   LockResource task(3)(3)    R2 3 to 1
63 UnlockResource task(3)(3)    R2 1 to 3
66      Completion      task( 3)( 3)    task( 1)( 2)          6               0               0
67   LockResource task(1)(2)    R1 9 to 8
68      Preemption      task( 1)( 2)    task( 2)( 2)
73      Completion      task( 2)( 2)    task( 1)( 2)          5               0               0
74 UnlockResource task(1)(2)    R1 8 to 9
76   LockResource task(1)(2)    R2 9 to 1
78 UnlockResource task(1)(2)    R2 1 to 9
79      Completion      task( 1)( 2)    task(63)             16               0               8
80      Preemption      task(63)        task( 3)( 4)
81   LockResource task(3)(4)    R2 3 to 1
83 UnlockResource task(3)(4)    R2 1 to 3
86      Completion      task( 3)( 4)    task(63)              6               0               0
94      Preemption      task(63)        task( 1)( 3)
95   LockResource task(1)(3)    R1 9 to 8
96      Preemption      task( 1)( 3)    task( 2)( 3)
100     Preemption      task( 2)( 3)    task( 3)( 5)
```

B. Description of implementation (10%)

首先，先在 ucosii.h 新增 locked Time , blocking time, OSEvevnt *R1,R2 (Mutex) 等等資料欄位。

```c
/*Task structure*/
typedef struct task_para_set {
    INT16U TaskID;
    INT16U TaskArriveTime;
    INT16U TaskExecutionTime;
    INT16U TaskPeriodic;
    INT16U TaskNumber;
    INT16U TaskPriority;
    INT16U R1_LockTime;
    INT16U R1_UnlockTime;
    INT16U R2_LockTime;
    INT16U R2_UnlockTime;

}task_para_set;
int TASK_NUMBER;
/*Task structure*/
```

```c
/*CPP */
INT8U R1_CEILING_PRIORITY;
INT8U R2_CEILING_PRIORITY;
/*CPP */
```

```c
/*resorce*/
OS_EVENT* R1; //resource R1
OS_EVENT* R2; //resource R2
/*resorce*/
```

```c
int holding_r1;
int holding_r2;

INT8U R1_LockTime;
INT8U R1_UnlockTime;
INT8U R2_LockTime;
INT8U R2_UnlockTime;

INT32U BlockingTime;
INT32U BlockingStartTick;
int  IsBlocked; // 1: blocked, 0: not blocked
```

並且在 main.c create Mutex：

```c
TraverseTCBList();

INT8U err;

R1 = OSMutexCreate(R1_CEILING_PRIORITY - 1, &err);
R2 = OSMutexCreate(R2_CEILING_PRIORITY - 2, &err);
```

修改 OSTCBInit 以支援新的資料欄位。

```
if (prio != OS_TASK_IDLE_PRIO) {
    task_para_set* task = &TaskParameter[id - 1];

    ptcb->remaining = 0;
    ptcb->period = task->TaskPeriodic;
    ptcb->execution_time = task->TaskExecutionTime;

    ptcb->ArriveTime = task->TaskArriveTime;
    ptcb->deadline = task->TaskArriveTime + task->TaskPeriodic;
    ptcb->state = 0; // created, not arrival
    ptcb->TaskNumber = 0;

    ptcb->R1_LockTime = task->R1_LockTime;
    ptcb->R2_LockTime = task->R2_LockTime;

    ptcb->R1_UnlockTime = task->R1_UnlockTime;
    ptcb->R2_UnlockTime = task->R2_UnlockTime;

    ptcb->holding_r1 = 0;
    ptcb->holding_r2 = 0;

    ptcb->TaskID = task->TaskID;

    ptcb->IsBlocked = 0;
    ptcb->BlockingStartTick = 0;
    ptcb->BlockingTime = 0;
}
```

接著，因為我想在 TimeTick 完成控制邏輯，所以新增 OSMutexAccept, pend ,post 以在 ISR 使用 mutex。 Mutex 使用 PCP 機制，即只有最高優先權且要使用此資源的 task 能鎖住 mutex。

```
BOOLEAN OSMutexAccept_ISR(OS_EVENT* pevent, OS_TCB* ptcb, INT8U* perr) {
    INT8U pcp;
    INT8U owner;

#if OS_CRITICAL_METHOD == 3u
    OS_CPU_SR cpu_sr = 0u;
#endif

#if OS_ARG_CHK_EN > 0u
    if (pevent == (OS_EVENT*)0 || ptcb == (OS_TCB*)0) {
        if (perr != (INT8U*)0) {
            *perr = OS_ERR_PEVENT_NULL;
        }
        return OS_FALSE;
    }
    if (pevent->OSEventType != OS_EVENT_TYPE_MUTEX) {
        if (perr != (INT8U*)0) {
            *perr = OS_ERR_EVENT_TYPE;
        }
        return OS_FALSE;
    }
#endif

    OS_ENTER_CRITICAL();
    pcp = (INT8U)(pevent->OSEventCnt >> 8u);
    owner = (INT8U)(pevent->OSEventCnt & 0xFF);

    if (owner == OS_MUTEX_AVAILABLE) {
        pevent->OSEventCnt &= OS_MUTEX_KEEP_UPPER_8;     // Clear LSB
        pevent->OSEventCnt |= ptcb->OSTCBPrio;           // Set task's priority
        pevent->OSEventPtr = (void*)ptcb;               // Mark ownership

        if ((pcp != OS_PRIO_MUTEX_CEIL_DIS) &&
            (ptcb->OSTCBPrio <= pcp)) {
            OS_EXIT_CRITICAL();
            if (perr != (INT8U*)0) {
                *perr = OS_ERR_PCP_LOWER;
            }
            return OS_TRUE; // Mutex was locked, but PCP violation
        }
        else {
            OS_EXIT_CRITICAL();
            if (perr != (INT8U*)0) {
                *perr = OS_ERR_NONE;
            }
            return OS_TRUE; // Success
        }
    }

    OS_EXIT_CRITICAL();
    if (perr != (INT8U*)0) {
        *perr = OS_ERR_NONE;
    }
    return OS_FALSE; // Mutex not available
}
```

```c
INT8U OSMutexPend_ISR(OS_EVENT* pevent, OS_TCB* ptcb) {
    INT8U pcp;
    INT8U owner;

#if OS_CRITICAL_METHOD == 3u
    OS_CPU_SR cpu_sr = 0u;
#endif

#if OS_ARG_CHK_EN > 0u
    if (pevent == (OS_EVENT*)0 || ptcb == (OS_TCB*)0) {
        return OS_ERR_PEVENT_NULL;
    }
    if (pevent->OSEventType != OS_EVENT_TYPE_MUTEX) {
        return OS_ERR_EVENT_TYPE;
    }
#endif

    OS_ENTER_CRITICAL();
    pcp = (INT8U)(pevent->OSEventCnt >> 8u);          // Extract ceiling
    owner = (INT8U)(pevent->OSEventCnt & 0xFFu);      // Extract owner prio

    if (owner == OS_MUTEX_AVAILABLE) {
        // Mutex is free: acquire it
        pevent->OSEventCnt &= OS_MUTEX_KEEP_UPPER_8;
        pevent->OSEventCnt |= ptcb->OSTCBPrio;
        pevent->OSEventPtr = (void*)ptcb;

        // Check PCP rule
        if ((pcp != OS_PRIO_MUTEX_CEIL_DIS) && (ptcb->OSTCBPrio <= pcp)) {
            OS_EXIT_CRITICAL();
            return OS_ERR_PCP_LOWER;
        }

        OS_EXIT_CRITICAL();
        return OS_ERR_NONE;  // Lock acquired
    }

    OS_EXIT_CRITICAL();
    return 0xFF;  // Already held
}
```

```c
INT8U OSMutexPost_ISR(OS_EVENT* pevent, OS_TCB* ptcb) {
    INT8U pcp;
    INT8U owner_prio;

#if OS_CRITICAL_METHOD == 3u
    OS_CPU_SR cpu_sr = 0u;
#endif

#if OS_ARG_CHK_EN > 0u
    if (pevent == (OS_EVENT*)0 || ptcb == (OS_TCB*)0) {
        return OS_ERR_PEVENT_NULL;
    }
    if (pevent->OSEventType != OS_EVENT_TYPE_MUTEX) {
        return OS_ERR_EVENT_TYPE;
    }
#endif

    OS_TRACE_MUTEX_POST_ENTER(pevent);

    OS_ENTER_CRITICAL();
    pcp = (INT8U)(pevent->OSEventCnt >> 8u);
    owner_prio = (INT8U)(pevent->OSEventCnt & 0xFFu);

    // Only the owner can release the mutex
    if ((OS_TCB*)pevent->OSEventPtr != ptcb) {
        OS_EXIT_CRITICAL();
        OS_TRACE_MUTEX_POST_EXIT(OS_ERR_NOT_MUTEX_OWNER);
        return OS_ERR_NOT_MUTEX_OWNER;
    }

    // Simply mark mutex as available (skip priority logic)
    pevent->OSEventCnt &= OS_MUTEX_KEEP_UPPER_8;
    pevent->OSEventCnt |= OS_MUTEX_AVAILABLE;
    pevent->OSEventPtr = (void*)0;

    OS_EXIT_CRITICAL();
    OS_TRACE_MUTEX_POST_EXIT(OS_ERR_NONE);
    return OS_ERR_NONE;
}
```

使用 resource_manger() 讓 task 需要或要釋放資源時，lock 或 unlock resource。

```c
void resource_manager(OS_TCB* ptcb) {
    INT32U now = ptcb->execution_time - ptcb->remaining;
    INT32U now_time = OSTimeGet();
    INT8U err;

    // ---------- Lock R1 (ISR-safe) ----------
    if (now == ptcb->R1_LockTime && !ptcb->holding_r1 && ptcb->R1_LockTime != 0) {
        INT8U original_prio = GetCurrentEffectivePriority(ptcb);
        INT8U ceiling = R1_CEILING_PRIORITY;
        INT8U owner_prio = R1->OSEventCnt & 0x00FF;

        INT8U accept_err;
        err = OSMutexAccept_ISR(R1, ptcb, &accept_err);
        if (accept_err == OS_ERR_NONE || accept_err == OS_ERR_PCP_LOWER) {
            ptcb->holding_r1 = 1;
            INT8U inherited_prio = GetEffectiveCeilingPriority(ptcb, ceiling, 1);

            fprintf(Output_fp, "%d   LockResource task(%d)(%d)\tR1 %d to %d\n",
                now_time, ptcb->TaskID, ptcb->TaskNumber, original_prio, inherited_prio);
            printf("%d   LockResource task(%d)(%d)\tR1 %d to %d\n",
                now_time, ptcb->TaskID, ptcb->TaskNumber, original_prio, inherited_prio);
        }
    }

    // ---------- Unlock R1 (ISR-safe) ----------
    if (now == ptcb->R1_UnlockTime && ptcb->holding_r1 && ptcb->R1_UnlockTime != 0) {
        INT8U pre_post_prio = GetCurrentEffectivePriority(ptcb);
        err = OSMutexPost_ISR(R1, ptcb);
        ptcb->holding_r1 = 0;
        INT8U restored_prio = GetCurrentEffectivePriority(ptcb);

        fprintf(Output_fp, "%d UnlockResource task(%d)(%d)\tR1 %d to %d\n",
            now_time, ptcb->TaskID, ptcb->TaskNumber, pre_post_prio, restored_prio);
        printf("%d UnlockResource task(%d)(%d)\tR1 %d to %d\n",
            now_time, ptcb->TaskID, ptcb->TaskNumber, pre_post_prio, restored_prio);
    }
```

```c
    // ---------- Lock R2 (ISR-safe) ----------
    if (now == ptcb->R2_LockTime && !ptcb->holding_r2 && ptcb->R2_LockTime != 0) {
        INT8U original_prio = GetCurrentEffectivePriority(ptcb);
        INT8U ceiling = R2_CEILING_PRIORITY;
        INT8U owner_prio = R2->OSEventCnt & 0x00FF;

        INT8U accept_err;
        err = OSMutexAccept_ISR(R2, ptcb, &accept_err);
        if (accept_err == OS_ERR_NONE || accept_err == OS_ERR_PCP_LOWER) {
            ptcb->holding_r2 = 1;
            INT8U inherited_prio = GetEffectiveCeilingPriority(ptcb, ceiling, 2);

            fprintf(Output_fp, "%d   LockResource task(%d)(%d)\tR2 %d to %d\n",
                now_time, ptcb->TaskID, ptcb->TaskNumber, original_prio, inherited_prio);
            printf("%d   LockResource task(%d)(%d)\tR2 %d to %d\n",
                now_time, ptcb->TaskID, ptcb->TaskNumber, original_prio, inherited_prio);
        }
    }

    // ---------- Unlock R2 (ISR-safe) ----------
    if (now == ptcb->R2_UnlockTime && ptcb->holding_r2 && ptcb->R2_UnlockTime != 0) {
        INT8U pre_post_prio = GetCurrentEffectivePriority(ptcb);
        err = OSMutexPost_ISR(R2, ptcb);
        ptcb->holding_r2 = 0;
        INT8U restored_prio = GetCurrentEffectivePriority(ptcb);

        fprintf(Output_fp, "%d UnlockResource task(%d)(%d)\tR2 %d to %d\n",
            now_time, ptcb->TaskID, ptcb->TaskNumber, pre_post_prio, restored_prio);
        printf("%d UnlockResource task(%d)(%d)\tR2 %d to %d\n",
            now_time, ptcb->TaskID, ptcb->TaskNumber, pre_post_prio, restored_prio);
    }
```

在遍歷 OSTCBList 時，呼叫 rescouce_manage 管理 R1,R2。然後判斷 blocked Time，如果有高優先權要跑卻不能跑時，blockedTime ++。

```
ptcb = OSTCBList;                                    /* Point at first TCB in TCB list               */
while (ptcb->OSTCBPrio != OS_TASK_IDLE_PRIO) {       /* Go through all TCBs in TCB list              */

    OS_ENTER_CRITICAL();
    //check resorce lock
    resource_manager(ptcb);

    //
    if (ptcb->OSTCBPrio < OSPrioCur && ptcb->state == 1) {
        ptcb->BlockingTime++;
    }

    //printf("task: %2d remaining : %2d\n", ptcb->TaskID,ptcb->remaining);
    // Deadline miss
    if (ptcb->state == 1 && ptcb->remaining > 0 && OSTime >= ptcb->deadline) {
        ptcb->state = 4;
    }
    // INT8U kkk = OSTime;
    if (ptcb->OSTCBDly != 0u) {                      /* No, Delayed or waiting for event with TO     */
        if(ptcb->state ==0)
            ptcb->OSTCBDly--;                            /* Decrement nbr of ticks to end of delay       */
        if (ptcb->OSTCBDly == 0u && ptcb->state == 0) {               /* Check for timeout
            ptcb->state = 5;
            if ((ptcb->OSTCBStat & OS_STAT_PEND_ANY) != OS_STAT_RDY) {
                ptcb->OSTCBStat &= (INT8U)~(INT8U)OS_STAT_PEND_ANY;   /* Yes, Clear status flag   */
                ptcb->OSTCBStatPend = OS_STAT_PEND_TO;                /* Indicate PEND timeout    */
            }
            else {
                ptcb->OSTCBStatPend = OS_STAT_PEND_OK;
            }

            if ((ptcb->OSTCBStat & OS_STAT_SUSPEND) == OS_STAT_RDY) {  /* Is task suspended?       */
                OSRdyGrp |= ptcb->OSTCBBitY;                 /* No,  Make ready          */
                OSRdyTbl[ptcb->OSTCBY] |= ptcb->OSTCBBitX;
                OS_TRACE_TASK_READY(ptcb);
            }
        }
    }
```

在 OS_SchedNew() 中實作 CPP 演算法，必須要小於等於 ceiling priority 才能鎖定 mutex，切換任務。

```
static  void  OS_SchedNew (void)
{
#if OS_LOWEST_PRIO <= 63u
    INT8U y;

    y = OSUnMapTbl[OSRdyGrp];
    OSPrioHighRdy = (INT8U)((y << 3u) + OSUnMapTbl[OSRdyTbl[y]]);

    // ---------- CPP: Prevent preemption if current task is protected by ceiling ----------
    if (OSTCBCur != NULL && (OSTCBCur->holding_r1 || OSTCBCur->holding_r2)) {
        INT8U cur_ceiling = OSTCBCur->OSTCBPrio;

        // Apply R1 ceiling if held
        if (OSTCBCur->holding_r1) {
            INT8U r1_ceiling = R1_CEILING_PRIORITY - 1;
            if (r1_ceiling < cur_ceiling) {
                cur_ceiling = r1_ceiling;
            }
        }

        // Apply R2 ceiling if held
        if (OSTCBCur->holding_r2) {
            INT8U r2_ceiling = R2_CEILING_PRIORITY - 2;
            if (r2_ceiling < cur_ceiling) {
                cur_ceiling = r2_ceiling;
            }
        }

        // Check ceiling constraint: only allow preemption if new task has truly higher priority
        if (cur_ceiling <= OSPrioHighRdy) {
            OSPrioHighRdy = OSPrioCur;  // Stay with current task
        }
    }

    // ---------- update ----------
    OSTCBHighRdy = OSTCBPrioTbl[OSPrioHighRdy];
```

印出完成訊息時要重新初始化 blocking time。

```
if (ptcb->TaskID > 0) {
    switch (ptcb->state) {
    case 2:
        PrintTask("Completion",NULL);
        ptcb->TaskNumber++;
        ptcb->state = 0;

        ptcb->BlockingStartTick = 0;
        ptcb->BlockingTime = 0;
        ptcb->IsBlocked = 0;
        break;
    case 3:
        //printf("%d Arrive task(%d)(job %d) t3\n", OSTime, ptcb->TaskID, ptcb->TaskNumber + 1);
        PrintTask("Completion",NULL);
        ptcb->ArriveTime = OSTime;
        ptcb->remaining = ptcb->execution_time;
        ptcb->TaskNumber++;
        ptcb->deadline = OSTime + ptcb->period;
        ptcb->state = 1;

        ptcb->BlockingStartTick = 0;
        ptcb->BlockingTime = 0;
        ptcb->IsBlocked = 0;

        break;
    case 4:
        //printf("%d MissDeadline task(%d)(job %d)\n", OSTime, ptcb->TaskID, ptcb->TaskNumber);
        Miss_ptcb = ptcb;
        break;
    case 5:
        //printf("%d Arrive task(%d)(job %d)t5\n", OSTime, ptcb->TaskID, ptcb->TaskNumber);
        ptcb->state = 1;
        ptcb->ArriveTime = OSTime;
        ptcb->deadline = OSTime + ptcb->period;
        ptcb->remaining = ptcb->execution_time;

        ptcb->BlockingStartTick = 0;
        ptcb->BlockingTime = 0;
        ptcb->IsBlocked = 0;
        break;
    default:
        break;
    }
}
```

Apphook.h 修改 InputFile() 讀取 R1,R2 資料欄位，並且根據週期 (RMS) 將他排序，然後填入 3 的倍數。然後算出每個資源的 ceiling priority，在 main 扣掉 resource index，create Mutex。

```c
void InputFile(void) {
    R1_CEILING_PRIORITY = 255;
    R2_CEILING_PRIORITY = 255;

    errno_t err;
    if ((err = fopen_s(&fp, INPUT_FILE_NAME, "r")) == 0) {
        printf("The file 'TaskSet.txt' was opened\n");
    }
    else {
        printf("The file 'TaskSet.txt' was not opened\n");
        return;
    }

    char str[MAX];
    char* ptr;
    char* pTmp = NULL;
    int TaskInfo[INFO];  // Expecting INFO >= 8
    int i, j = 0;
    TASK_NUMBER = 0;

    // --- Temporary mapping to sort by period ---
    typedef struct {
        int idx;
        int period;
    } TaskMap;
    TaskMap task_map[OS_MAX_TASKS];

    // --- Step 1: Read all task entries ---
    while (fgets(str, sizeof(str), fp)) {
        if (str[0] == '\n' || str[0] == '\0') continue;
        i = 0;
        ptr = strtok_s(str, " \t\r\n", &pTmp);
        while (ptr != NULL && i < INFO) {
            TaskInfo[i++] = atoi(ptr);
            ptr = strtok_s(NULL, " \t\r\n", &pTmp);
        }
        if (i < 8) continue;

        TaskParameter[j].TaskID = TaskInfo[0];
        TaskParameter[j].TaskArriveTime = TaskInfo[1];
        TaskParameter[j].TaskExecutionTime = TaskInfo[2];
        TaskParameter[j].TaskPeriodic = TaskInfo[3];  // Will use for RM
        TaskParameter[j].R1_LockTime = TaskInfo[4];
        TaskParameter[j].R1_UnlockTime = TaskInfo[5];
        TaskParameter[j].R2_LockTime = TaskInfo[6];
        TaskParameter[j].R2_UnlockTime = TaskInfo[7];

        task_map[j].idx = j;
        task_map[j].period = TaskInfo[3];

        TASK_NUMBER++;
        j++;
    }

    fclose(fp);
```

```
    fclose(fp);

    // Step 2: Sort tasks by period (RM): smaller period ¡÷ higher priority
    for (int x = 0; x < TASK_NUMBER - 1; x++) {
        for (int y = 0; y < TASK_NUMBER - 1 - x; y++) {
            if (task_map[y].period > task_map[y + 1].period) {
                // swap
                TaskMap temp = task_map[y];
                task_map[y] = task_map[y + 1];
                task_map[y + 1] = temp;
            }
        }
    }


    // --- Step 3: Assign priority = multiples of 3 (3, 6, 9, ...) ---
    for (int k = 0; k < TASK_NUMBER; k++) {
        int idx = task_map[k].idx;
        TaskParameter[idx].TaskPriority = (k + 1) * 3;  // priority: 3,6,9,...
        p2id[TaskParameter[idx].TaskPriority] = TaskParameter[idx].TaskID;
    }

    // --- Step 4: Compute Resource Ceiling Priorities ---
    for (int i = 0; i < TASK_NUMBER; i++) {
        int prio = TaskParameter[i].TaskPriority;

        if (TaskParameter[i].R1_LockTime > 0 || TaskParameter[i].R1_UnlockTime > 0) {
            if (prio < R1_CEILING_PRIORITY) {
                R1_CEILING_PRIORITY = prio;
            }
        }

        if (TaskParameter[i].R2_LockTime > 0 || TaskParameter[i].R2_UnlockTime > 0) {
            if (prio < R2_CEILING_PRIORITY) {
                R2_CEILING_PRIORITY = prio;
            }
        }
    }

    // Optional: Print results for debug
    printf("R1 ceiling priority = %d - 1 = %d\n", R1_CEILING_PRIORITY, R1_CEILING_PRIORITY - 1);
    printf("R2 ceiling priority = %d - 2 = %d\n", R2_CEILING_PRIORITY, R2_CEILING_PRIORITY - 2);
}
```

```
INT8U err;

R1 = OSMutexCreate(R1_CEILING_PRIORITY - 1, &err);
R2 = OSMutexCreate(R2_CEILING_PRIORITY - 2, &err);
```

[PART III] Performance Analysis [10%]

A. How it works

NPCS（Not preempted critical section）

原理：任務一旦進入 critical section (鎖定某些資源)，系統就不可被其他任務 preempted。即使有較高優先權的任務準備好，也必須等待當前持有資源的任務離開臨界區後才能執行。

運作方式：在 critical section 執行期間，其他任務無法搶奪 CPU。

CPP（Ceiling Priority Protocol）

原理：每個共享資源會設定一個 priority ceiling，等於所有會使用該資源的任務中最高的優先權。只有當任務的優先權高於目前系統中所有被鎖定資源的 ceiling 時，才允許鎖定資源，否則會被封鎖。

運作方式：任務在 lock 資源前會檢查目前的系統 ceiling。若沒鎖住或想鎖住的任務小於等於握有目前握有此資源的優先級，即可進入 critical section，並將自身優先權暫時提升為該資源的 ceiling。任務離開 critical section 時，恢復原本優先權。這樣的機制可防止高優先權任務被低優先權任務「長時間 blocked」，並且有效避免 deadlock。

B. Advantage

NPCS（Non-Preemptive Critical Section）

- 實作簡單，邏輯清楚，容易除錯與維護。
- 自然避免 deadlock，不需追蹤資源狀態。
- 不會產生優先權反轉（Priority Inversion）。
- Critical section 中不會有 context switch，切換成本低。

CPP（Ceiling Priority Protocol）

- 能有效避免 deadlock 與 priority inversion。
- 每個任務最多只會被 block 一次，且 block 時間可預測。
- 提供較高的系統 concurrency，允許非衝突任務繼續執行。
- 封鎖條件嚴謹，有明確的可驗證性，適合實時系統分析。
- 支援任務在 critical section 中動態提升優先權，確保資源快速釋放。

C. Disadvantage

NPCS（Non-Preemptive Critical Section）

- 系統整體 concurrency 低，進入 critical section 就會阻擋其他所有任務。
- 高優先權任務可能被低優先權任務長時間延遲（blocking time 不可預測）。
- 無法在 critical section 內插入重要的即時任務。

- 會拖延整體系統的 responsiveness。

CPP（Ceiling Priority Protocol）

- 實作較複雜，需要額外管理 resource ceiling 與任務狀態。
- 若資源分配與 ceiling 設計不當，可能導致任務過度被封鎖。
- 執行期間可能發生較多 context switch（任務優先權動態調整造成）。
- Ceiling 判斷機制在多資源複雜系統中，分析與驗證成本較高。